TABLE OF CONTENTS

Table of Contents i

Chapter 1 Mixture of Experts (MoE) 1
1.1 工作原理 1
1.2 模型格局（截至 2026 年 3 月） 1
1.3 DeepSeek 三大创新 2
1.3.1 细粒度专家（Fine-Grained Experts） 2
1.3.2 Auxiliary-Loss-Free 负载均衡 2
1.3.3 DeepEP：专用 All-to-All 通信库 2
1.4 源码关键路径 3

Chapter 2 Quantization（量化） 4
2.1 方法选择矩阵 4
2.2 H200 实测对比 4
2.3 GGUF K-Quant 深度解析 4
2.4 FP8 训练时量化 5

Chapter 3 Long Context（长上下文） 6
3.1 上下文长度排行 6
3.2 核心技术演进 6
3.2.1 YaRN（Yet Another RoPE extensioN） 6
3.2.2 iRoPE（Meta，LLaMA 4） 6
3.2.3 Ring Attention 7
3.3 Context Rot：最重要的研究发现 7
3.4 推理框架的长上下文支持 7

Chapter 4 Test-Time Compute（推理时计算缩放） 8
4.1 核心洞察 8
4.2 三种策略 8
4.3 推理模型格局 8
4.4 GRPO vs. REINFORCE++ vs. PPO 8
4.5 DeepSeek-R1 蒸馏管线 9

Chapter 5 Synthetic Data（合成数据） 10
5.1 前沿模型的合成数据策略 10
5.2 最优合成–真实数据比例 10
5.3 Model Collapse 风险 10

Chapter 6 Model Merging（模型合并） 11
6.1 核心方法 11

Chapter 7 推荐学习路径与资源 12

References 13

CHAPTER 1

MIXTURE OF EXPERTS (MOE)

MoE 架构已成为 2026 年所有 frontier 开源模型的标准选择。其核心价值在于以极低的训练成本获得同等质量的模型：DeepSeek-V3 以 $5.6M 的成本训练了 671B 参数模型，而西方同级别模型的训练成本高达数亿美元 [1].

1.1 工作原理

对于每个输入 token，MoE 层的处理流程如下：

1. **路由**：一个学习到的线性门控网络计算每个专家的分数。
2. **选择**：Top-K 选择激活 K 个专家（通常 $K = 2$ 或 $K = 8$）。
3. **并行计算**：被选中的专家独立处理 token。
4. **加权聚合**：按路由权重对专家输出求和。

相比于 dense 模型，MoE 使得总参数量远大于每个 token 实际激活的参数量。例如 DeepSeek-V3 总参数 671B，但每个 token 仅激活 37B（~5.5%）。

1.2 模型格局（截至 2026 年 3 月）

Table 1.1: 主要 MoE 模型对比

模型	总参数	活跃参数	专家数	Top-K	上下文
DeepSeek-V4	~1T	32B	256+	TBD	1M
DeepSeek-V3	671B	37B	256+1 shared	8	128K
Mistral Large 3	675B	41B	MoE	—	256K
LLaMA 4 Maverick	~400B	17B	128	custom	1M
Qwen3-235B	235B	22B	128	8	256K
GPT-OSS-120B	117B	5.1B	MoE+MXFP4	—	—
LLaMA 4 Scout	109B	17B	16	custom	10M

关键趋势：专家数量从 Mixtral 的 8 个猛增到 DeepSeek 的 256 个。更细粒度的专家意味着更灵活的组合路由——$\binom{256}{8} \approx 4.4 \times 10^9$ 种组合，相比 $\binom{8}{2} = 28$ 种，组合空间扩大了 10^8 倍。

1.3 DeepSeek 三大创新

1.3.1 细粒度专家（Fine-Grained Experts）

DeepSeek 使用 256 个路由专家加 1 个共享专家。每个路由专家仅为标准 FFN 的 1/32 大小，共享专家处理所有 token 以捕获通用知识：

$$\mathbf{y} = \text{SharedExpert}(\mathbf{x}) + \sum_{i \in \text{TopK}} g_i \cdot \text{Expert}_i(\mathbf{x}) \tag{1.1}$$

其中 g_i 为路由权重，TopK 选取 8 个专家。

1.3.2 Auxiliary-Loss-Free 负载均衡

传统方法在训练目标中加入辅助损失 L_{balance} 以惩罚专家利用率不均衡：

$$L_{\text{total}} = L_{\text{LM}} + \alpha \cdot L_{\text{balance}} \tag{1.2}$$

这引入了根本矛盾：α 太大损害模型质量，太小则专家坍缩。

DeepSeek 的方案：Bias-Based Balancing

不加辅助损失，而是给每个专家的路由分数加一个 **不参与反向传播** 的偏置项 b_i：

$$\text{score}_i = \text{softmax}\big(W_{\text{router}} \cdot \mathbf{x} + b_i\big)$$

偏置 b_i 由简单启发式规则在每步更新：专家过载则 $b_i \leftarrow b_i - \gamma$，空闲则 $b_i \leftarrow b_i + \gamma$（$\gamma \approx 10^{-3}$）。由于 b_i 不在计算图内，语言建模梯度完全不受干扰。2025 年芝加哥大学的理论论文证明该方法等价于 一步一迭代的原始–对偶分配问题求解，具有单调改进保证和对数遗憾界。

1.3.3 DeepEP：专用 All-to-All 通信库

MoE 需要跨 GPU 分发 token 到对应专家（all-to-all dispatch）。DeepEP 提供两种模式：

Table 1.2: DeepEP 两种内核模式

模式	用途	技术特点
Normal（高吞吐）	训练 + prefill	NVLink + RDMA 混合，SM 数量可控
Low-Latency	推理 decode	Hook-based 通信–计算重叠，不消耗 SM 资源

1.4 源码关键路径

MoE 前向传播的完整流程可在以下开源项目中追踪:

Table 1.3: 各框架 MoE 核心代码文件

框架	组件	关键路径
vLLM	路由器	`layers/fused_moe/router/fused_topk_router.py`
vLLM	融合内核	`layers/fused_moe/fused_moe.py` (79KB, Triton)
vLLM	DeepEP 后端	`deepep_ht_prepare_finalize.py`
Megatron	路由器	`moe/router.py` (TopKRouter)
Megatron	Token 调度	`moe/token_dispatcher.py`
Megatron	负载均衡损失	`moe/moe_utils.py`
DeepSpeed	分片 MoE	`moe/sharded_moe.py`
TorchTitan	EP 并行	`distributed/expert_parallel.py` (DTensor)
Nanotron	Grouped GEMM	`src/nanotron/nn/moe.py`

CHAPTER 2

QUANTIZATION（量化）

量化通过降低模型精度来减少内存占用、提高推理速度，同时控制质量损失在可接受范围内。

2.1 方法选择矩阵

量化方法速查

- **数据中心 GPU (H100/H200)**：FP8（vLLM/TRT-LLM），近零精度损失
- **Blackwell GPU (B200/RTX 5090)**：NVFP4，2.3× 吞吐提升
- **消费级 GPU + 高吞吐**：AWQ + Marlin kernel，741 tok/s（超过 FP16 的 461）
- **CPU / Apple Silicon**：GGUF Q4_K_M（llama.cpp / Ollama）
- **微调**：BitsAndBytes NF4（QLoRA）

2.2 H200 实测对比

在 H200 GPU 上对 Qwen2.5-32B-Instruct 进行 4-bit 量化的实测结果如 Table (2.1) 所示。

Table 2.1: H200 上 Qwen2.5-32B 各量化方法对比 [2]

方法	Perplexity	HumanEval	吞吐 (tok/s)
FP16 Baseline	6.56	56.1%	461
Marlin-AWQ	6.84	51.8%	**741**
BitsAndBytes	**6.67**	51.8%	168
GGUF Q4_K_M	6.74	51.8%	93

AWQ + Marlin 内核在 4-bit 下吞吐量反超 FP16（741 vs. 461 tok/s）。这是因为更小的模型减少了内存带宽瓶颈——量化不仅节省内存，还能提升推理速度。

2.3 GGUF K-Quant 深度解析

GGUF 的 K-quant 采用两级层级量化（hierarchical quantization）：

1. 层权重被切分为 **super-block**（如 256 个权重），配一个 FP16 缩放因子。
2. 每个 super-block 内部包含多个 **sub-block**（如 32 个权重），各自独立计算缩放因子并量化权重。
3. Sub-block 的缩放因子本身被**二次量化**（6-bit），由 super-block 级别的 FP16 缩放因子覆盖。

`Q4_K_M` 命名含义：4-bit 精度 / K-quant 方法 / Medium 精度变体。"M" 表示混合精度策略：attention 层的 Q/K/V 和 FFN down_proj 使用 Q6_K，其余层使用 Q4_K。仅多 ~0.2 bpw，perplexity 增量从 0.1149 降至 0.0535。

Table 2.2: GGUF 量化格式对比（基于 LLaMA 7B）

格式	有效 bpw	7B 大小	PPL Δ
Q8_0	8.0	6.7 GB	+0.0004
Q6_K	6.0	5.15 GB	+0.0044
Q5_K_M	5.1	4.45 GB	+0.0142
Q4_K_M	**4.5**	**3.80 GB**	**+0.0535**
Q3_K_M	3.3	3.06 GB	+0.2437
Q2_K	2.5	2.67 GB	+0.8698

2.4 FP8 训练时量化

DeepSeek-V3 是首个 FP8 端到端训练的大规模模型 [1]。核心技术包括：

1. **细粒度块级量化**：权重每 128×128 子矩阵独立缩放，激活每 1×128 子向量独立缩放。
2. **高精度累加**：FP8 Tensor Core 仅提供 14-bit 精度，DeepSeek 通过 4 次连续 WGMMA 操作后加入 FP32 累加器解决此问题。
3. **PyTorch 原生支持**：`torchao.float8` 的 rowwise recipe 在 2K H200 集群上实现 1.34–1.43× 训练加速。

CHAPTER 3

LONG CONTEXT（长上下文）

3.1 上下文长度排行

Table 3.1: 当前最长上下文窗口模型

模型	上下文	关键技术
LLaMA 4 Scout	10M	iRoPE（RoPE + NoPE 交替层）
Grok-4-fast	2M	未公开
Gemini 3 Pro	1M	Sparse MoE
Claude 4.5 Sonnet	1M	未公开
DeepSeek-V4	1M	MLA + DSA
Qwen2.5-1M	1M	YaRN + 渐进式预训练

3.2 核心技术演进

3.2.1 YaRN（Yet Another RoPE extensioN）

RoPE 在训练长度之外会" 外推失败"，位置编码进入未见过的频率空间。YaRN 整合了三个关键创新：

1. **NTK-By-Parts**：将频率维度分为三组——低频不动（保留原始模式）、中频 NTK-aware 缩放、高频激进缩放。
2. **Temperature Scaling**：在 softmax 中加入温度参数，短序列保持原始行为，长序列时稳定注意力分布。
3. **最小微调**：仅需原始预训练数据的极小比例即可实现巨大的上下文扩展。

目前 Qwen、DeepSeek、LLaMA 等主流模型均使用 YaRN 变体进行上下文扩展。

3.2.2 iRoPE（Meta，LLaMA 4）

LLaMA 4 交替使用两种注意力层：

- **RoPE 层**：带位置编码，捕捉局部上下文和相对位置。

- **NoPE 层**：无位置偏置，关注纯语义关系。

NoPE 层天然不受序列长度限制，而 RoPE 层通过 temperature scaling 在推理时泛化到超出训练长度，共同支撑了 LLaMA 4 Scout 的 10M token 上下文窗口。

3.2.3 Ring Attention

将长序列分割到多个 GPU 上，设备排列成环形拓扑，KV block 在设备间循环传递。每个设备计算自己持有的 query 与当前可见 KV 的注意力，通信与计算重叠以隐藏传输延迟。内存复杂度从 $O(n^2)$ 降为每设备 $O(n)$。

3.3 Context Rot：最重要的研究发现

Chroma 2025 年实证研究：18 个 frontier 模型

所有模型都随输入长度增加而性能下降，即使远未达到上下文窗口上限。

- **U 型曲线**：模型对序列开头和结尾的信息关注度最高，中间信息的检索准确度下降超过 30%。
- **反直觉发现**：有结构的上下文比打乱的上下文更难检索——模型在有逻辑的干扰信息中更容易”迷路”。
- **核心结论**：Context engineering（如何组织上下文）比 context length（能塞多少）更重要。

3.4 推理框架的长上下文支持

vLLM 提供 PagedAttention（KV cache 分页管理）、Continuous Batching、Chunked Prefill 和 Prefix Caching。

SGLang 的 Chunked Pipeline Parallelism（2026 年 1 月发布）在 DeepSeek-V3.1 上实测实现 3.31× prefill 吞吐提升，Qwen3-235B 处理 1M token 的 TTFT 从 55.5s 降至 10.5s（降低 81%）。

CHAPTER 4

TEST-TIME COMPUTE（推理时计算缩放）

4.1 核心洞察

推理时计算的颠覆性影响

7B 模型 + 100× 推理计算 ≈ 70B 模型标准推理。推理需求预计 2026 年超过训练需求 118 倍。GPT-4 等效性能成本从 2022 年 $20/M tokens 降至 2026 年 $0.40/M tokens （1000× 下降）。

4.2 三种策略

Table 4.1: 推理时计算缩放的三种策略

策略	方法	代表
Parallel	采样 N 个答案取共识	Best-of-N, Majority Vote
Sequential	链式推理，逐步验证	DeepSeek-R1 thinking tokens
Hybrid	并行 + 顺序结合	ThreadWeaver (1.53× 延迟降低)

4.3 推理模型格局

Table 4.2: 主要推理模型对比（2026 年初）

模型	特点
DeepSeek-R1-0528	671B MoE, AIME 87.5%, 开源 MIT
Qwen3	统一 thinking/non-thinking 模式，单模型两种用法
o3/o4-mini	首个推理 + 全工具访问，ARC-AGI 87%+
Claude Opus 4.6	可配置 thinking budget (32K/64K tokens)

4.4 GRPO vs. REINFORCE++ vs. PPO

这三种算法是训练推理模型的核心 RL 方法。

GRPO（Group Relative Policy Optimization）的核心思想是 消除 PPO 中的 critic network，使用组内采样估算 advantage：

$$A_i = \frac{R_i - \text{mean}(\{R_j\})}{\text{std}(\{R_j\})} \tag{4.1}$$

$$J_{\text{GRPO}} = \mathbb{E}\left[\min\left(\frac{\pi_\theta}{\pi_{\text{old}}} A_i,\ \text{clip}\left(\frac{\pi_\theta}{\pi_{\text{old}}},\ 1 \pm \epsilon\right) A_i\right)\right] - \beta \cdot D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \tag{4.2}$$

Table 4.3: 三种 RL 算法对比

维度	**PPO**	**GRPO**	**REINFORCE++**
Advantage 估计	Critic network $V(s)$	组内 reward 均值/标准差	组内均值 + batch-level 标准差
内存需求	4 份模型副本	3 份（无 critic），内存减半	同 GRPO
稳定性	最稳定	存在 reward collapse	更鲁棒
可扩展性	70B+ 极难	可行	可行

2026 年共识：GRPO 最流行（DeepSeek-R1 验证），REINFORCE++ 更稳定（逐步替代），PPO 在 70B+ 因内存限制几乎不可行。

4.5 DeepSeek-R1 蒸馏管线

DeepSeek-R1 的完整训练经历五个阶段：

1. **Base**：DeepSeek-V3-Base（671B MoE 预训练模型）。
2. **纯 RL**：使用 GRPO 训练 → R1-Zero。自然涌现 chain-of-thought、自我验证、aha moments。
3. **Cold-start SFT + 二轮 RL**：先用精选数据做 SFT，再做第二轮 RL → R1。
4. **蒸馏**：R1 生成 ~800K 推理 trace，SFT 训练小模型 → R1-Distill-Qwen-1.5B/7B/14B/32B, R1-Distill-Llama-8B/70B。
5. **对齐**：安全对齐和行为稳定。

开源复现项目 HuggingFace Open-R1 (19.9K stars) 完整实现了上述三阶段方法。

CHAPTER 5

SYNTHETIC DATA（合成数据）

合成数据已成为 frontier 模型训练的标准组件。2026 年的共识策略是**合成 + 精选人类数据的混合方法**。

5.1 前沿模型的合成数据策略

Table 5.1: 前沿模型的合成数据使用方式

模型	策略
Phi-4 (Microsoft)	仅 140 万精选样本 SFT，证明数据质量 > 数据规模
Qwen3	36T tokens 含大量用 Qwen2.5-Math/Coder 生成的领域合成数据
DeepSeek-R1	纯 RL 训练推理能力，再用 R1 推理 trace 蒸馏到小模型

5.2 最优合成–真实数据比例

Meta FAIR 训练超过 1000 个 LLM 的大规模研究得出核心结论：

Meta FAIR 研究结论

预训练最优比例：~30% 合成 + ~70% 真实数据，可加速训练 5–10×。

- 单轮 rephrased 合成数据不会导致 model collapse。
- 递归生成和纯 textbook 风格数据会导致 collapse。
- 8B 参数的生成器产出效果不亚于更大模型。

5.3 Model Collapse 风险

在合成数据上递归训练会导致模型" 遗忘" 数据分布的尾部，逐代性能退化。缓解策略是始终混入精选的人类/网络数据，典型比例为 50–80% 合成 + 20–50% 人类数据。

CHAPTER 6

MODEL MERGING（模型合并）

模型合并将多个微调后的模型组合成一个，无需额外训练。仅用 CPU（无需 GPU）即可通过 MergeKit 实现。

6.1 核心方法

Table 6.1: 模型合并方法对比

方法	原理	备注
SLERP	球面线性插值	两个模型平滑混合
TIES-Merging	修剪 + 符号选举 + 合并	处理参数冲突
DARE	丢弃 90–99% task vector 权重	极致稀疏化
Task Arithmetic	加减 task vector	简单而有效
进化式合并	进化算法搜索最优配置	2025 年最大突破 (Sakana AI)

2025 年的最大突破来自 Sakana AI 的**进化式模型合并**, 发表于 Nature Machine Intelligence, 使用进化算法搜索最优合并配置（层级、权重系数），已集成到 MergeKit 中。

MergeKit（Arcee AI，6.8K stars）是该领域无可争议的标准工具，CPU 即可运行。NeurIPS 2024 举办了正式的 LLM Merging Competition，标志着模型合并已从实验技术进入主流。

CHAPTER 7

推荐学习路径与资源

Table 7.1: 按主题推荐的核心阅读资源

主题	优先级	资源
MoE	必读	Cameron Wolfe: nanoMoE (手写 PyTorch 实现)
MoE	必读	Sebastian Raschka: Big LLM Architecture Comparison
MoE	推荐	Shirley Li: DeepSeek-V3 Explained (8 篇系列)
量化	必读	JarvisLabs: vLLM 量化完全指南 (H200 实测)
量化	必读	Colfax Research: DeepSeek FP8 训练
量化	推荐	llama.cpp PR #1684 (K-quant 原始实现)
长上下文	必读	Chroma: Context Rot (18 模型实证)
长上下文	必读	Aman Arora: How LLMs Scaled from 512 to 2M Context
长上下文	推荐	LMSYS: SGLang Pipeline Parallelism
推理计算	必读	Lilian Weng: Why We Think
推理计算	必读	The Illustrated GRPO
推理计算	推荐	Cameron Wolfe: GRPO++
合成数据	推荐	Meta FAIR: 1000+ LLM 合成数据实验
模型合并	推荐	Maxime Labonne: Merge Models with MergeKit

REFERENCES

[1] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024 (cit. on pp. 1, 5).

[2] JarvisLabs, *VLLM quantization complete guide and benchmarks*, 2025. [Online]. Available: `https://docs.jarvislabs.ai/blog/vllm-quantization-complete-guide-benchmarks` (cit. on p. 4).